Segurança em SSO via OIDC

Quarkus avançado

Matheus Cruz

SSO

O que significa SSO?

Significa Single-Sign-On (*Ou Login Único*)

Imagina que você entra na sua conta Google. Depois disso, você consegue acessar automaticamente:

- Gmail
- YouTube
- Google Drive

Isso tudo sem precisar digitar a senha novamente. Isso é SSO 💡

SSO

Como funciona o SSO?

O SSO funciona com base em:

- Um provedor de identidade (responsável por autenticar o usuário)
- Vários serviços (que confiam nesse provedor)

Protocolos comuns mais utilizados:

- SAML
- **OAuth 2.0**
- **OpenID Connect**

SSO

Benefícios do SSO

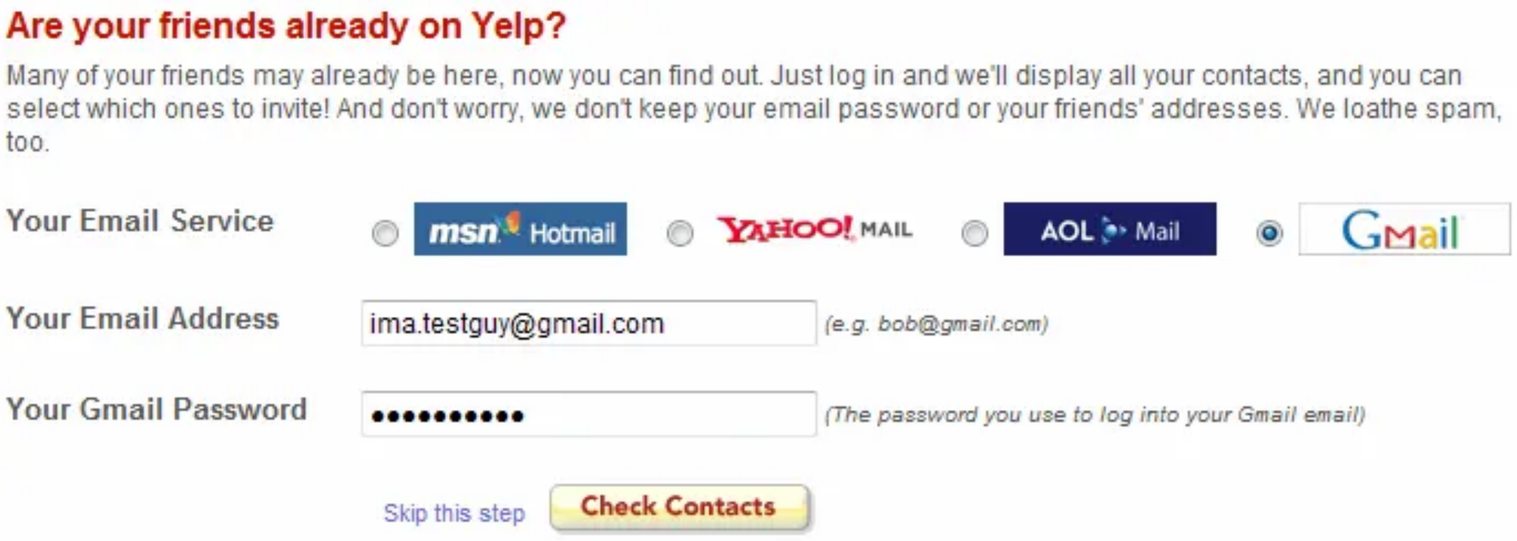
- Melhor experiência do usuário (menos logins)
- Centralização da autenticação
- Mais controle de segurança (políticas únicas, MFA, etc.)

Quem viabiliza o SSO é o OAuth 2.0 + OpenID Connect

OAuth 2.0

OAuth 2.0

Como era o mundo antes do OAuth 2.0



OAuth 2.0 é ***sobre delegação de acesso***

OAuth 2.0

Atores

Resource Owner: É o dono do recurso (meus contatos)

Resource Server: É quem detém o recurso (Gmail)

Client: É quem quer ter acesso ao recurso (Facebook)

Authorization Server: É quem por meio do consentimento do **Resource Owner** vai autorizar o acesso ao recurso (os contatos do **Resource Owner**)

OAuth 2.0

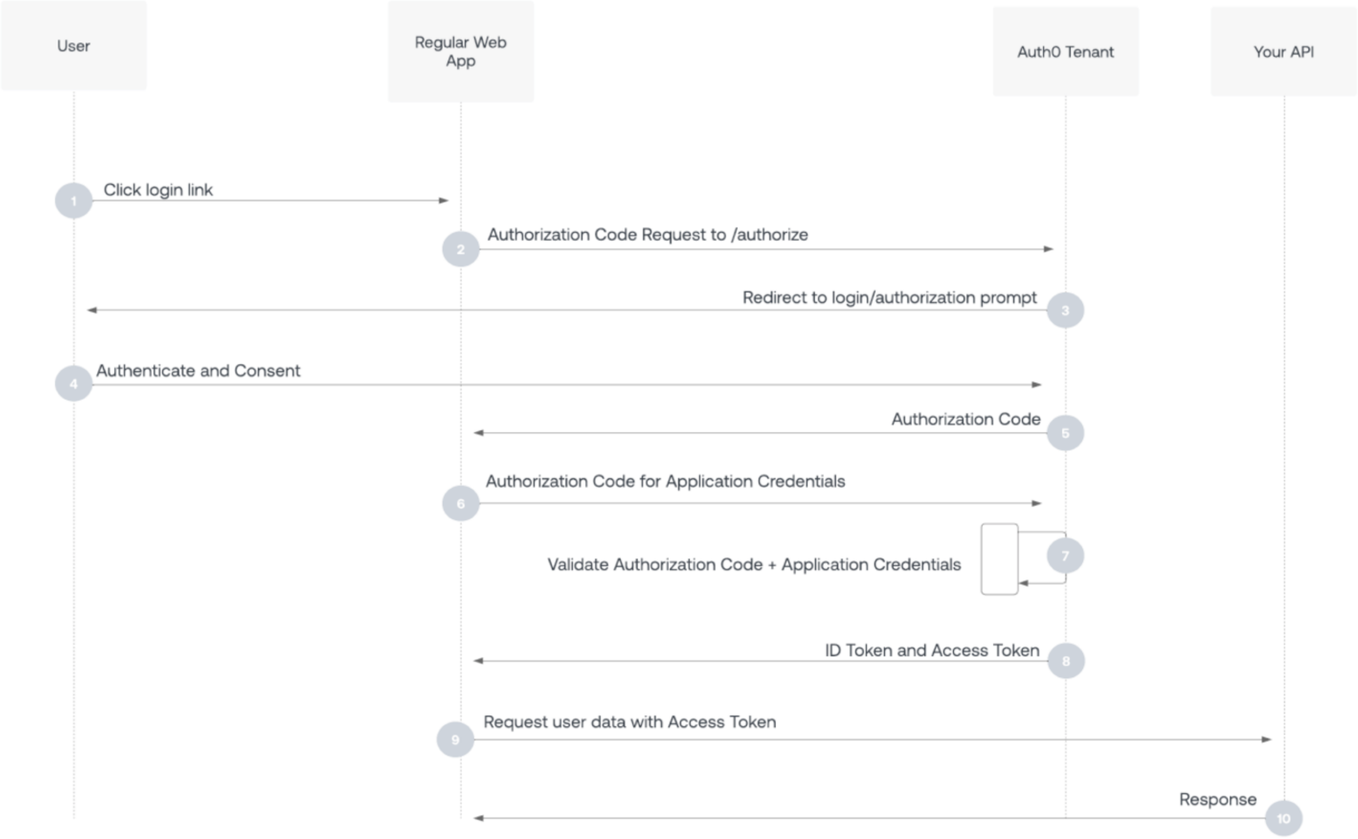
Clientes

OAuth 2.0

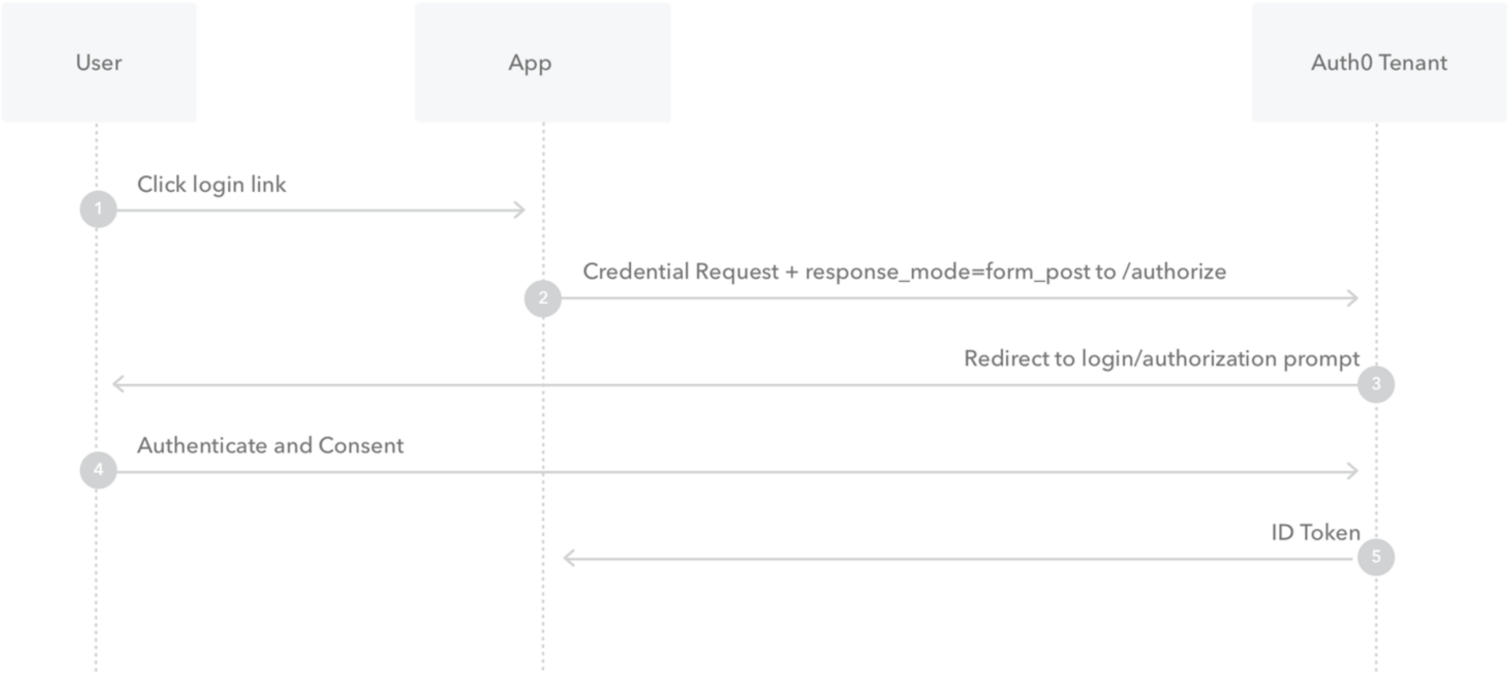
Fluxos

- Authorization Code Flow
- Implicit Flow
- Client Credentials Flow
- Authorization Code + PKCE

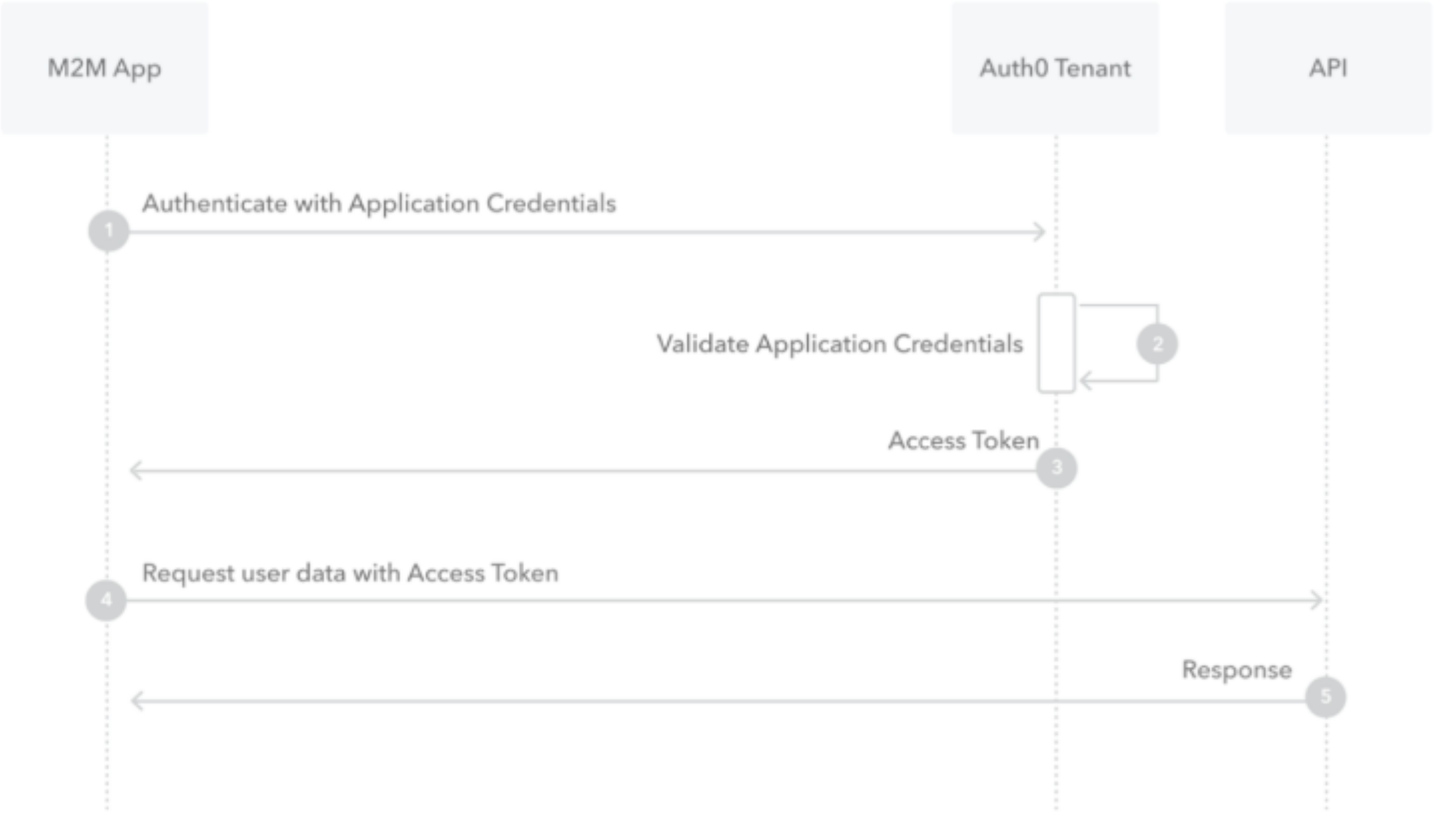
Authorization Code Flow



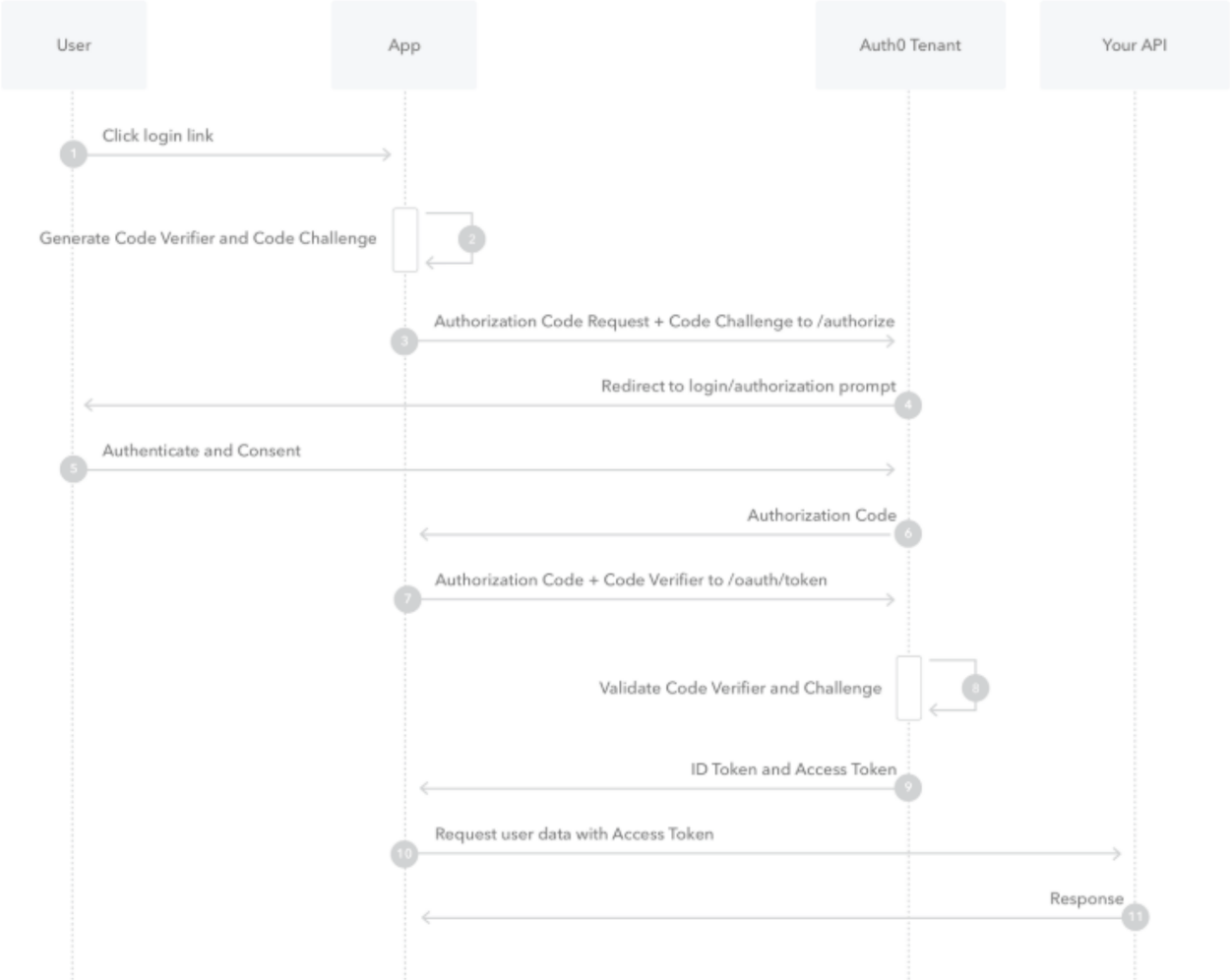
Implicit Flow



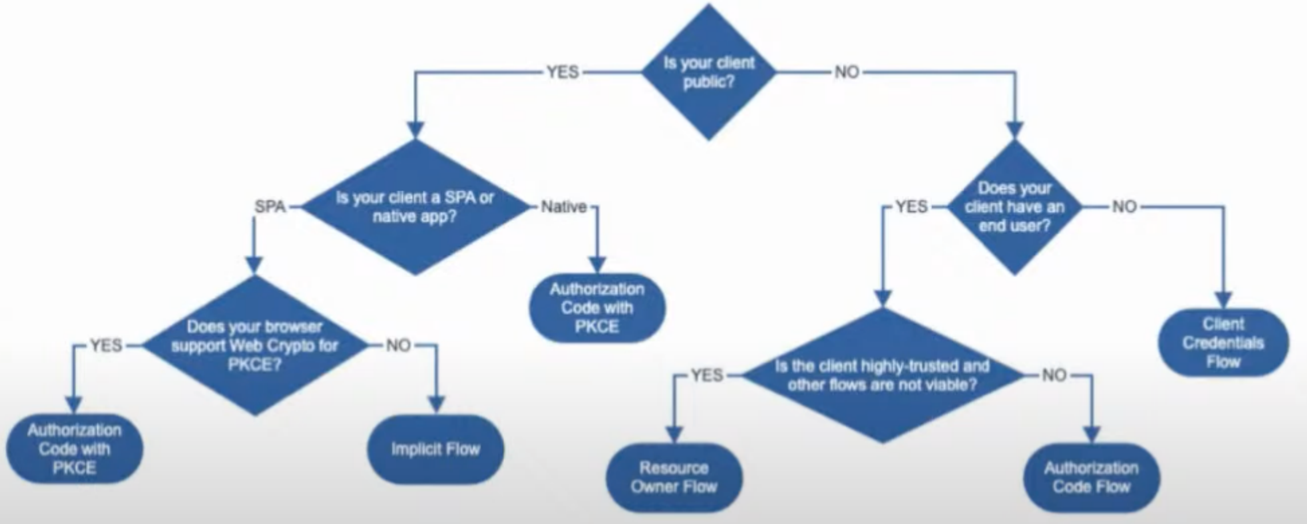
Client Credentials Flow



Authorization Code Flow + PKCE



Como escolher o fluxo correto?



OpenID Connect

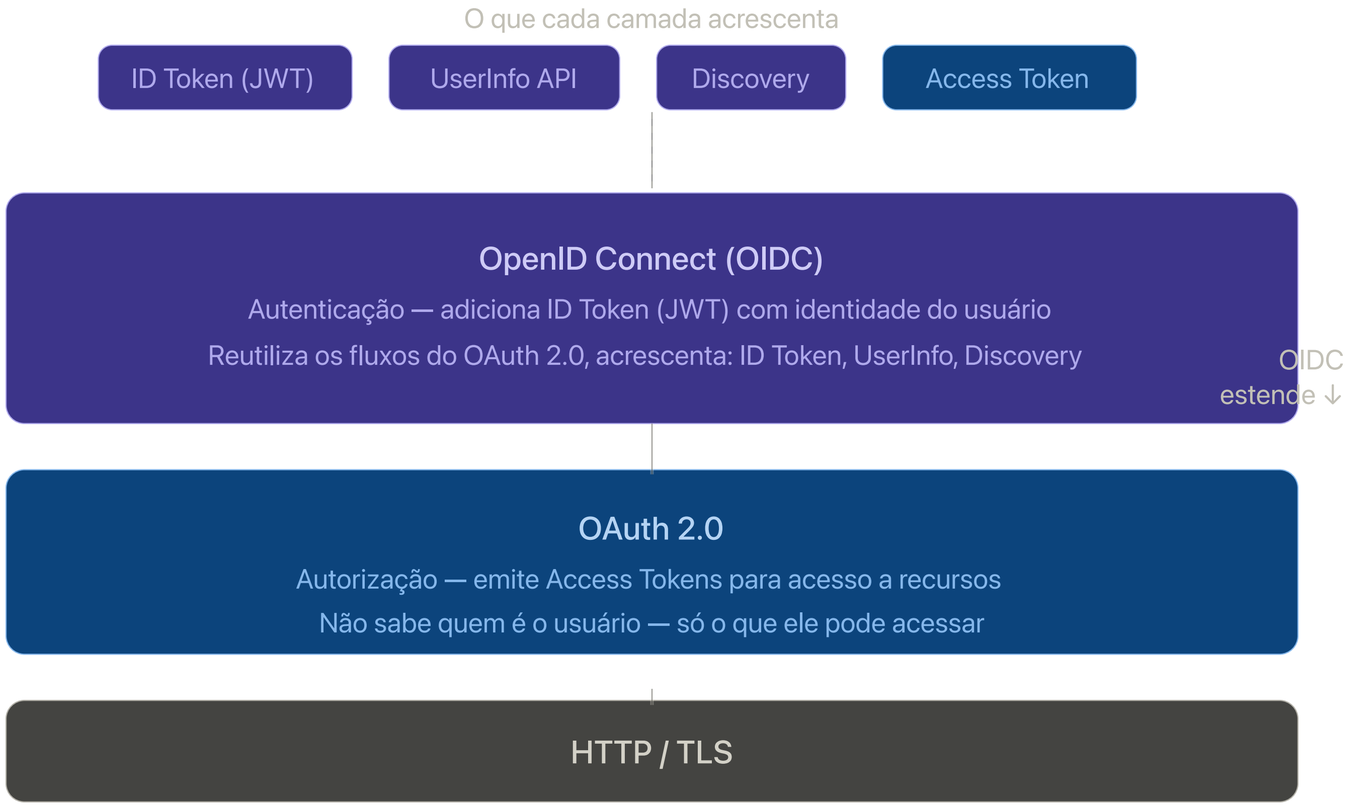
É um protocolo de autenticação construído em cima do OAuth 2.0.

Ele permite que aplicações verifiquem a identidade de um usuário e obtenham informações básicas do seu perfil de forma padronizada e segura.

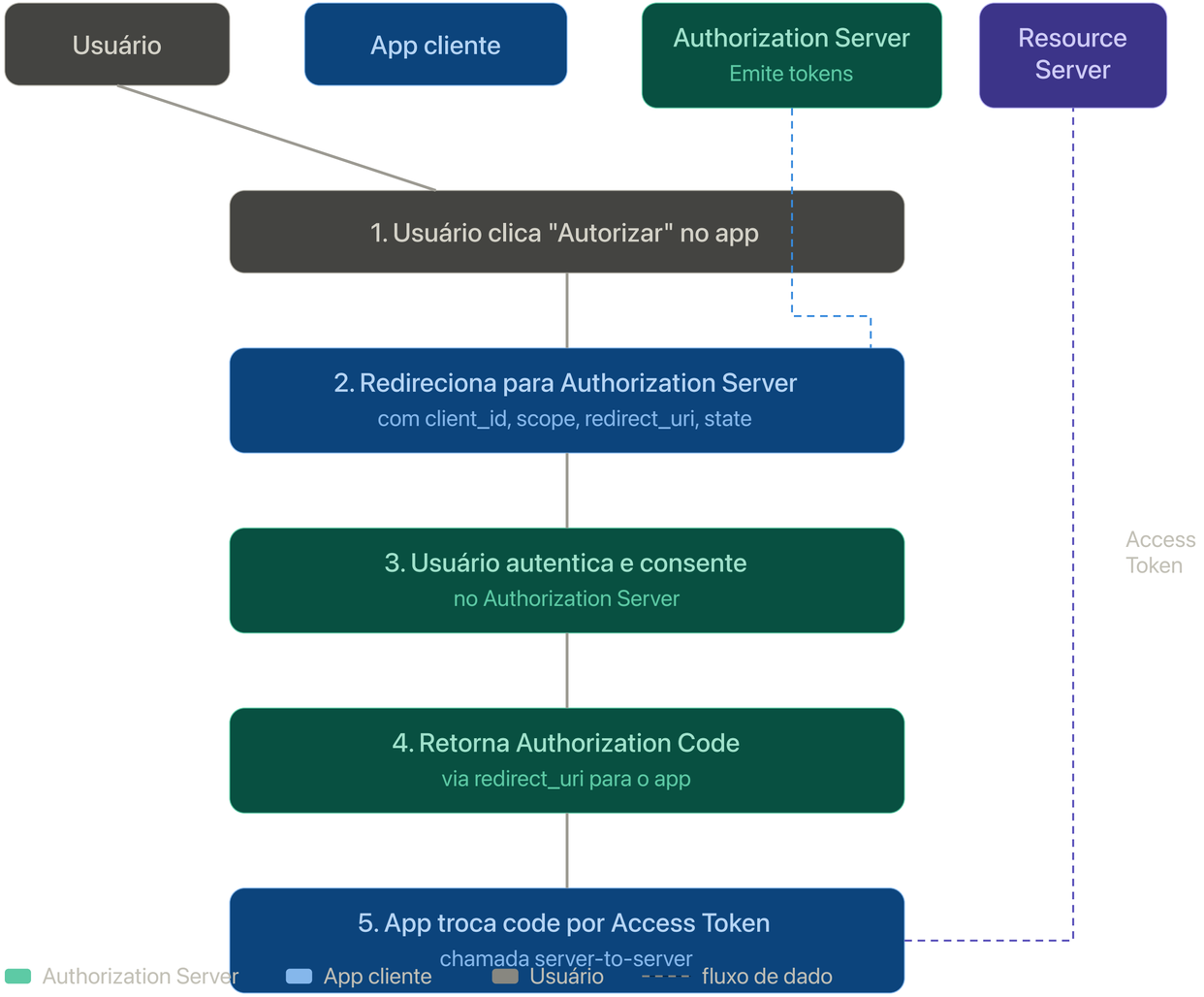
O OAuth 2.0 não é sobre autenticação, é sobre autorização, consentimento e delegação, eu como usuário autorizo o **cliente** a fazer coisas no meu lugar.

Por mais que tenha autenticação no meio, pois o usuário precisa se identificar junto ao Identity Provider o OAuth 2.0 é sobre autorização.

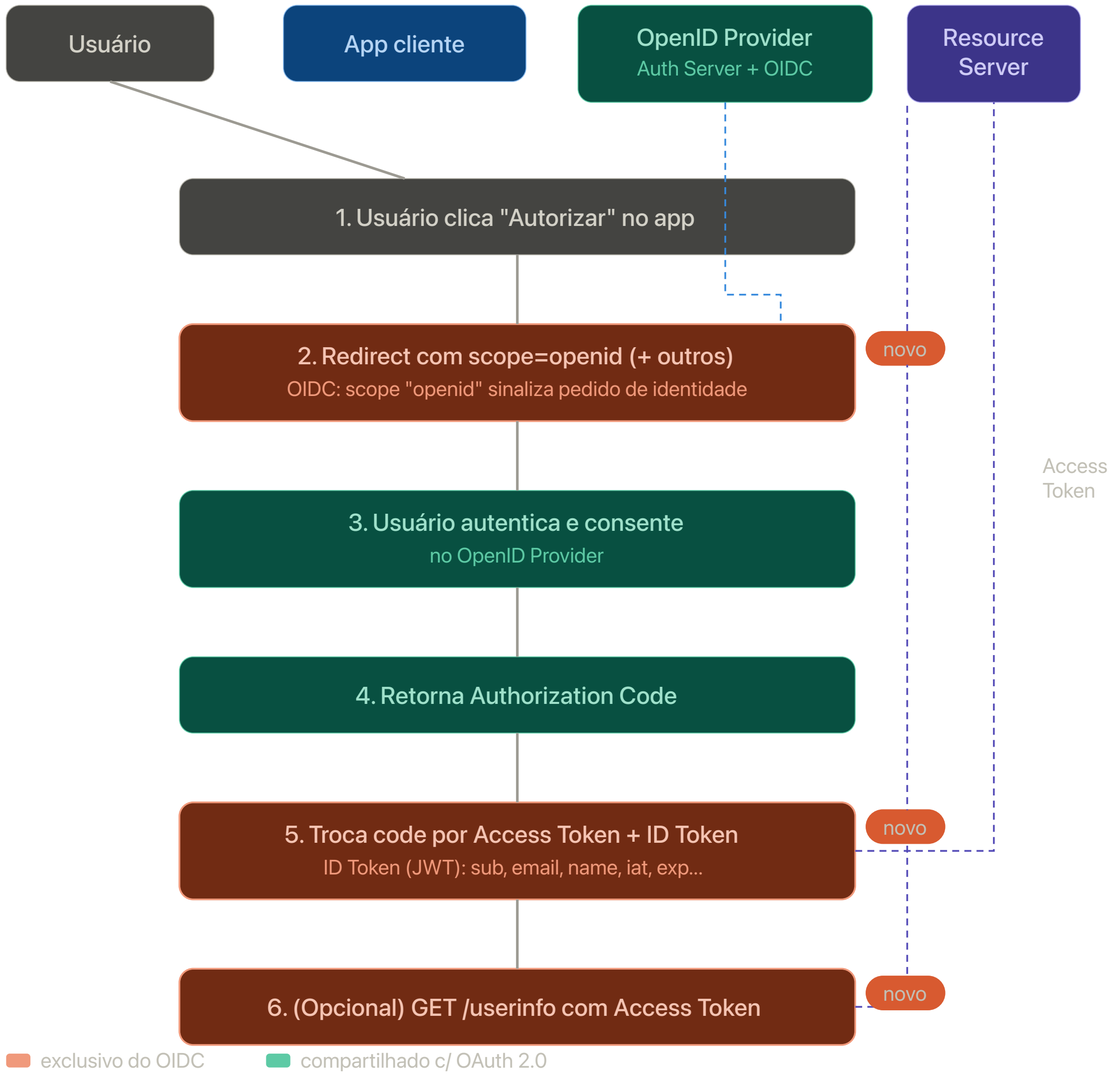
OAuth 2.0 + OIDC



OAuth 2.0


Usuário
App cliente
Authorization Server
Emite tokens
Resource Server
1. Usuário clica "Autorizar" no app
2. Redireciona para Authorization Server
com client_id, scope, redirect_uri, state
3. Usuário autentica e consente
no Authorization Server
4. Retorna Authorization Code
via redirect_uri para o app
5. App troca code por Access Token
chamada server-to-server
Access Token
Authorization Server
App cliente
Usuário
fluxo de dado

OAuth 2.0 + OIDC



Keycloak

- Projeto Open Source
- Feito em Java e as novas versões são baseadas no Quarkus
- Serve como um identity provider
- Serve como o nosso **Authorization Server**
- Se integra facilmente com **Github, Gmail, Facebook**, etc.

Quarkus OIDC

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-oidc</artifactId>
</dependency>
```